DevSecOps: SAST, SCA & DAST Scanning

A user guide to automated security scanning for the Meridian (myblogs) project

Every pull request into main runs three categories of automated security scanning, defined in `.github/workflows/pr-check.yml` ("PR Checks") alongside the ordinary build/test jobs:

- **SAST** (Static Application Security Testing) — analyzes this project's own source code, without running it (Section 1).
- **SCA** (Software Composition Analysis) — inspects the third-party packages it depends on, not its own code (Section 2).
- **DAST** (Dynamic Application Security Testing) — tests the application while it's actually running, by interacting with it like a black-box attacker would (Section 3).

Each category is covered by more than one tool — not for redundancy, but because each tool catches a different slice of the problem, explained in its own section below. Section 4 explains SARIF, the file format that gets some (not all) of these tools' results onto GitHub's Security tab, and Section 5 is a short reference table if you just want the at-a-glance version.

1. SAST — Static Application Security Testing

1.1 What is SAST?

SAST analyzes a project's **own source code** without running it, looking for bugs and exploitable security flaws — injection, XSS, hardcoded secrets, insecure crypto — directly in the code a developer wrote. It never touches third-party packages; that's SCA's job (Section 2). Two SAST tools run here, covering different depths of the same first-party code:

- **SonarQube** — broad code quality + security: bugs, vulnerabilities, code smells, security hotspots, duplication, and a coverage-gated Quality Gate.
- **CodeQL** — narrower but deeper: real interprocedural dataflow/taint-tracking specifically for security vulnerabilities, with fewer false positives on the bug classes it targets.

> *Why both, not one: SonarQube's vulnerability detection is rule/pattern-based, which is fast and broad but can miss or over-flag findings that depend on how data actually moves through the program. CodeQL's dataflow analysis is slower and narrower in scope, but traces that movement precisely. Running both means whichever style catches a given bug catches it — neither tool alone covers what the other does.*

1.2 SonarQube

What it is & what it's used for. A self-hosted static analysis platform that parses source (and ingests test coverage reports you provide) and evaluates the result against a configurable set of rules and thresholds called a **Quality Gate**. It checks four categories: **Bugs** (Reliability), **Vulnerabilities** (Security), **Code Smells** (maintainability), and **Security Hotspots** (security-sensitive code a human must manually confirm is or isn't exploitable in context). Alongside issues, it also tracks **test coverage** and **duplicated code density** as gate conditions.

> *SonarQube Community Edition (what this project uses) does not scan third-party dependencies for known CVEs — no SCA capability at all. That gap is exactly what Section 2's tools cover.*

Configuring on GitHub. Nothing to enable in repo Settings — SonarQube is self-hosted, not a GitHub-native feature. The only GitHub-side piece is three **repository secrets** (`SONAR_TOKEN`,

SONAR_HOST_URL, SONAR_PROJECT_KEY) that the CI job injects as environment variables — without them the scan/gate steps in `pr-check.yml` fail outright, since there is no server to talk to.

Configuring in code. Three files, each with one job:

- `sonar-project.properties` (repo root) — what gets scanned, independent of secrets: `sonar.sources` (scan root), `sonar.exclusions` (keeps node_modules/, dist/, *.db, uploads/, coverage/, .github/ out of analysis), `sonar.javascript.file.suffixes` / `sonar.typescript.file.suffixes` (includes .vue as analyzable), and the two coverage report-path settings (Section 1.2, coverage below).
- `package.json` scripts — `sonar` runs the scan (the `@sonar/scan` npm package's `sonar-scanner` binary — no Java install or Docker image needed), `sonar:gate` checks the result, `sonar:check` chains both.
- `scripts/sonar-quality-gate.sh` — the actual gate logic (below), called identically by both a developer's machine and CI so neither ever runs different logic than the other can reproduce.

> *sonar.projectKey=${SONAR_PROJECT_KEY} inside sonar-project.properties is a literal placeholder — SonarScanner only substitutes ${env.VAR_NAME}-style references. The real key is supplied explicitly on the command line instead (below), so this placeholder has no functional effect.*

How it's done here. Locally, with `.env` populated with real SONAR_* values (loaded via `dotenv-cli`, the same tool the project's `start` scripts already use):

```
npm run sonar        # run the scan only, uploads results to the server
npm run sonar:gate   # check the gate result of the most recent scan
npm run sonar:check  # run both in sequence; exits non-zero if the gate fails
npm run sonar:full    # coverage:all + sonar:check — the whole CI pipeline, locally
```

In CI, defined in `.github/workflows/pr-check.yml` ("PR Checks") as a job alongside build-check, CodeQL, Dependency Review, and OWASP ZAP (Sections 1.3, 2.3, 3.2) — all in the same file, all triggered by the same `on: pull_request: branches: [main]` block, so one PR runs every check this repo has. The secrets come from GitHub Actions repository secrets instead of .env; dotenv-cli silently no-ops when no .env file is present, so the identical npm commands run unchanged in both places:

```
sonarqube:
  permissions: {pull-requests: read}
  steps:
    - actions/checkout@v4  (fetch-depth: 0)
    - actions/setup-node@v4
    - actions/setup-python@v5  (3.11)
    - run: pip install -e ".[all]"   # pytest + every agent extra
    - run: npm ci --ignore-scripts
    - run: npm run coverage:all   # 4 services + frontend + Python
    - name: SonarQube Scan
      env: {SONAR_TOKEN, SONAR_HOST_URL, SONAR_PROJECT_KEY}  # from GitHub Secrets
      run: npm run sonar
    - name: SonarQube Quality Gate check
      env: {SONAR_TOKEN, SONAR_HOST_URL, SONAR_PROJECT_KEY}
      run: npm run sonar:gate
```

This job is what actually blocks a PR merge on a failed gate (via a required status check), which running from a developer's machine or IDE alone cannot enforce. The Python setup step exists because `coverage:all` shells out to pytest for the meridian_agents suite — without it, that step fails with "No module named pytest", tolerated only because it runs with `continue-on-error: true`.

As a complementary, faster feedback loop, the VS Code extension **SonarQube for IDE** (SonarLint) can connect to the same server (Connected Mode) so issues show up inline while typing, using the server's quality profile. It's local-only, though — it never uploads results or evaluates the gate; a supplement to the scan-and-gate flow, not a replacement for it.

Gate mechanics — scripts/sonar-quality-gate.sh

Analysis and gate evaluation are asynchronous, in two stages:

- **Scan (client-side):** sonar-scanner analyzes source locally, writes an intermediate report under `.scannerwork/`, and uploads it via `POST /api/ce/submit`. The server responds with a task receipt written to `.scannerwork/report-task.txt` (ceTaskId, dashboardUrl).
- **Processing (server-side):** the upload is queued as a Compute Engine (CE) background task, which ingests the report, computes final measures, matches issues against the active quality profile, and evaluates the Quality Gate — all before the task reports SUCCESS.

The gate script then:

- Reads `ceTaskId` and polls `GET /api/ce/task?id=...` every 10s (up to 30 attempts) until it reaches SUCCESS/FAILED/CANCELED — failing immediately if it never reaches SUCCESS, since the gate would otherwise reflect a stale prior analysis.
- Queries `GET /api/qualitygates/project_status?projectKey=...` — this reads an already-computed result, it does not trigger new evaluation.
- Prints the overall status (SonarQube's API uses `ERROR` for a failed gate, not `FAIL`) and, on failure, each failing condition's actual value and threshold, then exits non-zero — what lets CI block the PR on this step.

Example: a real gate failure observed on this project

Default gate conditions evaluate only *new* code (lines changed since the gate's baseline — the "leak period"), not the whole codebase. This example is from a scan after a large batch of test files was added in one session, so "new code" spans everything since that baseline:

Metric	Actual	Required	Meaning
new_coverage	19.0%	≥ 100%	Coverage on lines changed since baseline
new_duplicated_lines_density	5.27%	≤ 3%	Too much duplicated new code
new_security_hotspots_reviewed	0.0%	= 100%	Hotspot(s) not yet manually reviewed
new_violations	73	= 0	New bugs / vulnerabilities / code smells introduced

A failing gate does not mean the scan failed — the scan always succeeds if it can reach the server and upload results. Pass/fail is purely the gate's evaluation of those results against its configured thresholds.

> *The new_coverage threshold shown here (≥100%) is unusually strict versus SonarQube's default "Sonar way" gate (typically ≥80%) — worth reviewing in the SonarQube UI if the standard threshold was actually intended. The other three conditions are independent of coverage entirely.*

Code coverage — how it feeds the gate

Coverage is a runtime metric, not a static-analysis one: as tests execute, a coverage tool records which lines actually ran. It measures only whether a line *executed*, not whether the test asserted anything

meaningful about it — necessary but not sufficient for a trustworthy suite, while low coverage reliably signals completely unverified code. The gate's coverage condition is scoped to new/changed code specifically so it acts as a forcing function while the author still has full context, not months later during a bug report.

SonarQube doesn't execute tests itself — it's purely a consumer of reports the project's own test runners produce, declared in sonar-project.properties: `sonar.javascript.lcov.reportPaths` (one lcov.info per JS/TS/Vue codebase) and `sonar.python.coverage.reportPaths` (one Cobertura XML for meridian_agents). During analysis the scanner cross-references each report's line numbers against the files it parsed and derives per-file, per-directory, whole-project, and git-diff-scoped (`new_coverage`) percentages — the last of which the gate actually evaluates.

Codebase	Runner	Command	Report
auth-service, blog-service, media-service, api-gateway	Jest (ts-jest)	npm run test:cov:<service> (jest --coverage)	coverage/lcov.info
frontend	Vitest (v8 provider)	npm run test:cov:frontend (vitest run --coverage)	coverage/lcov.info
meridian_agents (Python)	pytest + pytest-cov	npm run coverage:python (--cov-report=xml)	coverage-python.xml

> *Coverage reports must already exist on disk before npm run sonar runs — the scan is a point-in-time read. Stale or missing reports silently produce 0% or a previous run's numbers, not an error.* `npm run coverage:all` *regenerates every report in one command;* `npm run sonar:full` *chains coverage:all + the scan + the gate.*

1.3 CodeQL

What it is & what it's used for. GitHub's own SAST engine: it treats source as a queryable relational database and performs real interprocedural **dataflow / taint-tracking** analysis — tracing how a specific piece of untrusted input actually flows into a dangerous sink (a SQL query, a shell exec, an HTML response), rather than matching syntactic patterns. This is what makes it materially stronger than pattern-based tools at finding real, exploitable injection / XSS / SSRF / insecure-deserialization vulnerabilities with fewer false positives — at the cost of being narrower in scope than SonarQube (Section 1.2): no code smells, no duplication, no coverage gate.

Configuring on GitHub. Runs as native **GitHub Code Scanning** — free for public repositories (this repo is public), with zero additional infrastructure: no server to host, no secret to manage, unlike SonarQube's self-hosted setup. Results surface on the repo's **Security tab** → **Code scanning alerts**. The workflow job needs a `permissions: {security-events: write}` block to upload results there — everything else about enabling it lives in the workflow file itself, no repo Settings toggle required.

Configuring in code. Two files:

- `.github/codeql/codeql-config.yml` — enables the `security-extended` query pack (broader than the default query set) and excludes node_modules/, dist/, coverage/, uploads/, and DB/PDF files from analysis, deliberately mirroring SonarQube's `sonar.exclusions` so both tools scan the same real source.
- `.github/workflows/pr-check.yml` — the `codeql` and `codeql-gate` jobs (below).

> *CodeQL analysis initially lived in its own .github/workflows/codeql.yml — GitHub's own default convention (the "Set up code scanning" button always generates a standalone file). It was folded into pr-check.yml instead for one concrete reason: the gate job needs to know when analysis has finished, and jobs can only declare a native needs: dependency on a job within the same workflow file — not across two independent workflow runs. Keeping it separate would have meant polling GitHub's Checks API from the gate job just to work around that file boundary. A job's permissions: block is still scoped per-job regardless of which file it lives in, so this didn't widen any other job's permissions — the one real trade-off is losing CodeQL's independent trigger set (it also ran on every push to main and a weekly schedule as a standalone workflow); in one shared file, every job uses the same pull_request-only trigger.*

How it's done here. The single `on: pull_request: branches: [main]` block at the top of pr-check.yml governs every job in the file — CodeQL runs whenever a PR targets main, exactly like build-check and the SonarQube scan:

```
codeql:
  permissions: {security-events: write, actions: read, contents: read}
  strategy:
    matrix:
      language: [javascript-typescript, python]
  steps:
    - actions/checkout@v4
    - actions/setup-node@v4  (js/ts leg only)
    - run: npm ci --ignore-scripts  (js/ts leg only)
    - github/codeql-action/init@v3
      with: {languages: matrix.language,
          config-file: .github/codeql/codeql-config.yml}
    - github/codeql-action/autobuild@v3
    - github/codeql-action/analyze@v3

codeql-gate:
  needs: codeql    # waits for both matrix legs to finish
  permissions: {security-events: read, contents: read}
  steps:
    - actions/checkout@v4
    - run: npm run codeql:gate
```

The matrix runs two independent legs in parallel, one per language. The javascript-typescript leg installs dependencies first purely to help CodeQL resolve TS/JS imports more precisely; neither language actually needs a compiled build to be analyzed, so `autobuild` is effectively a no-op for both.

Custom severity gate — scripts/codeql-quality-gate.sh

`codeql-action/analyze` always exits successfully regardless of what it finds — GitHub Code Scanning has no built-in "fail the build on severity count" gate the way SonarQube's Quality Gate does natively. This script closes that gap, mirroring sonar-quality-gate.sh's approach rather than reusing SonarQube's exact thresholds:

- **FAIL** if any open alert has security severity **CRITICAL**.
- **FAIL** if more than **2** open alerts have security severity **HIGH**.
- otherwise **PASS**.

Because `codeql-gate` declares `needs: codeql`, it only starts once every matrix leg has completed — no polling required to know analysis is done. The one remaining timing wrinkle: a finished analyze step means the SARIF upload was accepted, not that GitHub has finished turning it into queryable alerts yet, so the script pauses briefly before querying `GET /repos/{owner}/{repo}/code-scanning/alerts?state=open` (paginated), filtering each alert's `rule.security_severity_level`.

> *An empty result from that query is genuinely ambiguous — zero open alerts, or alerts not finished processing yet — so the script uses a fixed pause rather than retrying-until-nonempty, which would silently treat "not ready yet" as "clean".*

1.4 SonarQube vs CodeQL — why both run, not either/or

Both tools are SAST and both report Vulnerabilities, which invites the question of whether one makes the other redundant. It doesn't — each has strengths the other doesn't attempt:

CodeQL's edge over SonarQube	SonarQube's edge over CodeQL
Real interprocedural dataflow/taint-tracking — traces how a specific untrusted input reaches a dangerous sink, not just pattern-matching the code around it. Fewer false positives on the injection/XSS/SSRF-class bugs it targets.	Test **coverage** as a first-class gate condition — CodeQL has no concept of coverage at all (Section 1.2, coverage subsection).
Free, zero infrastructure for public repos — no server to host, no secret to manage.	Much broader issue categories: **Code Smells** (maintainability), **duplicated code density**, and general **Bugs** beyond security — none of which are in CodeQL's scope at all.
security-extended query pack is purpose-built for security; nothing to configure beyond enabling it.	**Security Hotspots** — flags security-sensitive code (crypto, cookies, deserialization) for a human to triage, instead of only binary-flagging it as a vulnerability or not.
Native GitHub Code Scanning — results land directly on the Security tab via SARIF (Section 4), no separate UI to check.	Configurable, tunable rule sets and quality profiles on a real dashboard — better suited to enforcing house style/maintainability standards over time, not just point-in-time security findings.

> *Net effect used here: SonarQube is the general quality gate (bugs, smells, duplication, coverage-gated); CodeQL adds a deeper, security-specific pass on top of it. Running both means whichever style of analysis catches a given bug, catches it.*

2. SCA — Software Composition Analysis

2.1 What is SCA?

SCA inspects the **third-party packages** a project depends on — not its own source — against databases of publicly known vulnerabilities (CVEs) and license issues. Neither SonarQube Community Edition nor CodeQL do this (Section 1); a vulnerable dependency sitting in node_modules or a Python virtualenv is completely invisible to both, since both only parse first-party source. SCA closes that gap by matching exact package+version pairs already recorded in the project's **dependency graph** (derived from lockfiles/manifests) against an advisory database — fundamentally a *lookup*, not an analysis, so it needs no understanding of how the code that uses the package actually works.

Two tools cover this here, at different scopes:

- **Dependabot** — continuous, whole-dependency-tree coverage, independent of any PR.
- **Dependency Review** — scoped to exactly what a single PR's diff introduces, with an inline gate.

2.2 Dependabot

"Dependabot" is a brand name covering **three** genuinely distinct GitHub features, easy to conflate because they share one config file and one Security-tab presence. Each is explained separately below because each has a different job and a different reason for existing.

- **2.2.1 Dependabot alerts** — the actual SCA lookup: notices when a dependency already in use has a known vulnerability.
- **2.2.2 Dependabot security updates** — automatically opens a PR to *fix* a dependency an alert just flagged.
- **2.2.3 Dependabot version updates** — opens PRs to bump dependencies to their latest version on a schedule, vulnerable or not; not a security feature by itself.

> *GitHub deliberately keeps these independent: security-update PRs are never grouped with version-update PRs, and dependabot.yml (2.2.3) has no effect on whether alerts or security updates happen — the only link between them is that merging a security-update PR automatically closes the alert that triggered it.*

2.2.1 Dependabot alerts

What it is & the need for it. The actual SCA tool (Section 2.1's "lookup, not analysis" description is specifically this). It's *event-driven*, not scheduled: it re-checks whenever the dependency graph changes (a manifest/lockfile push) *or* whenever a new advisory is published for a package already in use — meaning an alert can appear on a dependency nobody has touched in months, the moment a CVE for it becomes public. Without this, a vulnerable dependency already merged into the codebase would sit there silently forever; nothing else in this document watches the tree continuously the way this does.

Configuring on GitHub. A repo Settings toggle: **Settings** → **Code security and analysis** → **Dependabot alerts**. Nothing to configure in code — confirmed enabled on this repo (the API's `/vulnerability-alerts` endpoint returns 204). Results surface on the repo's **Security tab** → **Dependabot alerts**, entirely independent of PR activity.

How it's done here. Enabled, nothing further to configure. Real example observed on this repo: two alerts appeared on `multer` (used by media-service and api-gateway) — high severity for a deeply-nested-field-names DoS, medium for incomplete cleanup of aborted uploads — the moment those advisories were published, with zero commits made to this repo at that time. That's the event-driven behavior above, not a coincidence of timing.

2.2.2 Dependabot security updates

What it is & the need for it. Alerts (2.2.1) only *notify* — something still has to act on them. When this is enabled, Dependabot automatically tries to open a pull request for every open alert that has an available patch: it checks whether the vulnerable dependency can be upgraded to the minimum version containing the fix without breaking the dependency graph, then raises a PR to exactly that version and links it to the alert. This is a completely separate mechanism from version updates (2.2.3) — it needs no `dependabot.yml` entry at all to function, since it's driven by alerts, not by a schedule.

Configuring on GitHub. Its own repo Settings toggle, separate from alerts: **Settings** → **Code security and analysis** → **Dependabot security updates**. Confirmed enabled on this repo (the API's `security_and_analysis.dependabot_security_updates.status` field reads `enabled`).

How it's done here. Enabled, nothing further to configure — this is the mechanism behind any PR titled like a plain version bump but opened *without* waiting for Monday's schedule (2.2.3), and whose merge closes a specific alert rather than just updating a version number.

2.2.3 Dependabot version updates

What it is & the need for it. Opens scheduled PRs bumping dependencies to their latest version, vulnerable or not — *not* a security feature; it exists to stop the codebase drifting arbitrarily far behind upstream, since an old-enough dependency eventually becomes hard to patch safely at all,

security-relevant or not.

Configuring on GitHub. No Settings toggle — committing `.github/dependabot.yml` to the repo is itself what enables it, per ecosystem entry.

Configuring in code. `.github/dependabot.yml` — one entry per package ecosystem:

```
updates:
  - package-ecosystem: npm
    directory: /
    schedule: {interval: weekly, day: monday, time: "06:00"}
    open-pull-requests-limit: 10
  - package-ecosystem: pip
    directory: /
    schedule: {interval: weekly, day: monday, time: "06:00"}
    open-pull-requests-limit: 5
```

> *Deliberately no groups: — each bump becomes its own individual PR so the maintenance agent can parse a plain "Bump X from A to B" title and decide whether to merge or close it, rather than having to unpack a batched multi-package PR.*

How it's done here. One npm entry and one pip entry, both rooted at `/` — a direct consequence of this repo's single-root-package.json consolidation (previously five separate npm entries, one per service, before that restructuring). Weekly on Mondays, capped at 10 open npm PRs / 5 open pip PRs at a time.

So which mechanism opened that PR I just saw?

Two different, independent things create Dependabot PRs on this repo, and the title/body tells you which:

- If it references a **security advisory / Dependabot alert** in its description, or appeared without waiting for a Monday — that's a **security update** (2.2.2), reacting to an alert. Merging it closes that alert.
- If it's a routine "Bump X from A to B" with no advisory reference, opened on/near Monday 06:00 UTC — that's a **version update** (2.2.3), running on dependabot.yml's schedule regardless of vulnerability status.

2.3 Dependency Review Action

What it is & what it's used for. A GitHub Action that runs the same SCA lookup as Dependabot alerts (same underlying Advisory Database) but scoped to exactly what a single PR's diff changes in the dependency manifests — and, unlike Dependabot alerts, with a **native inline gate**: it can fail the check itself, rather than only posting an informational alert to the Security tab. It complements Dependabot alerts rather than replacing them: Dependabot alerts cover the whole existing tree continuously; this action only ever sees what a given PR is about to add.

Configuring on GitHub. Requires **Dependency graph** enabled (**Settings** → **Code security and analysis** → **Dependency graph**) — a real gap hit while wiring this up on this repo: the first run failed outright with "Dependency review is not supported on this repository. Please ensure that Dependency graph is enabled", even though Dependabot alerts (2.2) were already active. Turning the toggle on and re-running the same job with no code changes fixed it. No other GitHub-side setup is needed — no secret, no additional permission beyond the job's own `contents: read`.

Configuring in code. One job in `.github/workflows/pr-check.yml`:

```
dependency-review:
  permissions: {contents: read}
  steps:
    - actions/checkout@v4
    - actions/dependency-review-action@v4
      with: {fail-on-severity: high}
```

How it's done here. `fail-on-severity: high` blocks the PR if the diff introduces anything rated high *or* critical — a plain binary cutoff, unlike the "0 critical, ≤2 high" *count*-based tolerance used for the CodeQL and SonarQube gates (Section 1). That count-based tolerance exists because those gates evaluate the whole existing codebase, where some accumulated, already-triaged risk is tolerated; a single PR realistically introduces at most one or two new dependencies at a time, so there's no equivalent "accumulated" risk to tolerate here — any new high/critical dependency is worth blocking outright. First real run on this repo passed clean (no vulnerable or license-problematic new dependencies).

3. DAST — Dynamic Application Security Testing

3.1 What is DAST?

DAST tests the application **while it is actually running**, from the outside — sending it real HTTP requests and observing the responses, the way an attacker (or a browser) would, with no access to source code at all. This is the fundamental difference from Sections 1 and 2: SAST and SCA are both static — they read files (source, or dependency manifests) and never execute anything. DAST catches an entirely different class of problem as a result: things that only exist at runtime — missing security headers, cookies without the right flags, information disclosed in error responses, endpoints reachable that shouldn't be — regardless of how clean the underlying source looks to a static tool.

> *The trade-off is depth of understanding: a DAST tool has no idea what the code is supposed to do, only how it actually responds to being poked. It complements SAST/SCA rather than substituting for either — this project runs all three.*

3.2 OWASP ZAP

What it is & what it's used for. OWASP ZAP (Zed Attack Proxy) is a free, open-source DAST tool. It runs as a proxy that **spiders** a target — following every link/form it can discover, building a map of the application — then inspects the actual HTTP traffic that generates against a ruleset of known-bad patterns. ZAP has two very different modes:

- **Baseline (passive) scan** — spiders the app and inspects whatever responses that spidering naturally generates: headers, cookies, disclosed information. It never submits a form or sends a crafted attack payload, so it cannot mutate data and is safe to run against a live-ish instance.
- **Full (active) scan** — additionally sends real attack payloads (SQL injection strings, XSS payloads, etc.) at every form/endpoint the spider found, including ones that create/delete real data. Far more thorough, but slower, noisier, and will happily submit every form it finds.

This project runs the **baseline** scan — it needs no authentication flow, no throwaway test data, and can't corrupt the seeded content already committed to blog.db/auth.db/media.db, which made it the reasonable default to start from.

Configuring on GitHub. Nothing to enable in repo Settings, unlike CodeQL — ZAP is not a GitHub-native feature, it's a third-party GitHub Action (`zaproxy/action-baseline`) that runs the official ZAP Docker image. No secret required for the scan itself. Its results currently do *not* reach the Security tab at all (Section 4) — they live only in the workflow log and a build artifact.

Configuring in code. One job in `.github/workflows/pr-check.yml`:

```
dast:
  permissions: {contents: read}
  steps:
    - actions/checkout@v4
    - actions/setup-node@v4
    - run: npm ci
    - name: Start the app
      run: |
        npm start & wait-loop curl-ing :5173 and :3000/api/posts
    - zaproxy/action-baseline@v0.15.0
      with: {target: http://localhost:5173,
          fail_action: false, allow_issue_writing: false,
          artifact_name: zap-baseline-report}
```

How it's done here. The app is booted via `npm start` directly on the runner — not the real production Dockerfile. That image also builds the TTS service (PyTorch + Kokoro model download), which DAST doesn't need at all, so building it would add several minutes to every PR for no scanning benefit; raw `npm start` boots the whole app in roughly 15 seconds. The step backgrounds it (`&`) and polls both the frontend (port 5173) and the API gateway (port 3000) with `curl` until both respond, up to 30 attempts. ZAP then targets `http://localhost:5173` — the actual browsable SPA, not the api-gateway's bare JSON API directly, since a passive spider needs crawlable HTML/links to find the attack surface at all.

> *GitHub-hosted Linux runners execute Docker-based actions (which is what action-baseline is, under the hood) with host networking, so "localhost" inside ZAP's container really does mean the runner's own localhost where npm start is listening — this only holds on Linux runners, which is what this workflow uses throughout.*

This job has no gate — `fail_action: false` means it cannot fail the PR no matter what it finds, and `allow_issue_writing: false` stops it from filing a GitHub issue every run (its default behavior, which would otherwise need `issues: write` permission this job deliberately doesn't have). This is intentional for now: no baseline run has happened yet to know what this app's normal finding count even looks like. Turning on a hard gate blind — the way SonarQube's and CodeQL's gates only were, after seeing real results (Sections 1.2, 1.3) — would risk blocking every future PR on pre-existing findings unrelated to whatever it actually changes. Results are only visible today via the `zap-baseline-report` build artifact and the job's own log.

4. SARIF — what actually reaches the Security tab

SARIF (Static Analysis Results Interchange Format) is a standard JSON schema for representing static-analysis findings — which tool found what, where, how severe, with what confidence — so different tools and different consumers (like GitHub's Security tab) can speak the same language instead of every tool needing its own bespoke integration. GitHub's **Code scanning alerts** UI is fundamentally a SARIF viewer: anything that gets a SARIF file to `github/codeql-action/upload-sarif` (or an equivalent upload call) shows up there, and nothing else does.

Of every tool in this document, only one currently does that:

Tool	Generates SARIF?	Reaches the Security tab?
CodeQL	Yes — built into codeql-action/analyze	Yes — automatically, same step (Code scanning alerts)
SonarQube	No	No — findings live on the separate, self-hosted SonarQube server UI

Tool	Generates SARIF?	Reaches the Security tab?
Dependabot alerts	No	Yes — but via GitHub's own advisory- matching, a different pipeline entirely (Dependabot alerts, not Code scanning)
Dependency Review	No	No — inline PR annotation only, not a persistent Security-tab entry
OWASP ZAP	Can, but not configured here	Not currently — see below

> *ZAP can emit a SARIF-format report (there are documented options/community actions for this), which could then be pushed to the Security tab with an additional github/codeql-action/upload-sarif step — exactly the mechanism CodeQL uses internally. That extra step isn't wired up in this repo yet (Section 3.2); today ZAP's results are only visible via its build artifact and the job log. Worth revisiting once the DAST job has a calibrated gate (Section 3.2) and is trusted enough to make its findings persistent alongside CodeQL's.*

Two consequences worth being explicit about: first, "check GitHub's Security tab" does not mean "check everything" — SonarQube and (for now) ZAP both require checking their own separate places. Second, Dependabot alerts and Code scanning alerts are two different sub-tabs fed by two entirely different pipelines that happen to live under the same Security tab umbrella, not one unified feed.

5. Summary

Tool	Category	Scope	Trigger	Blocks merge?
SonarQube	SAST	Whole repo (gate: new/changed code)	pull_request (pr-check.yml)	Yes — sonar:gate
CodeQL	SAST	Whole repo (JS/TS + Python matrix)	pull_request (pr-check.yml)	Yes — custom codeql-gate
Dependabot alerts	SCA	Whole dependency tree, continuous	Event-driven (new advisory / graph change)	No — informational, Security tab only
Dependabot security updates	SCA (remediation)	Whole dependency tree, continuous	Event-driven (reacts to an alert)	N/A — opens PRs, doesn't gate
Dependabot version updates	(not security)	Whole dependency tree	Weekly schedule	N/A — opens PRs, doesn't gate
Dependency Review	SCA	This PR's manifest diff only	pull_request (pr-check.yml)	Yes — fail-on-severity: high
OWASP ZAP	DAST	Running app, passive spider	pull_request (pr-check.yml)	No — fail_action: false (pending baseline data)

Generated for the Meridian (myblogs) project.